A MINOR PROJECT

REPORT ON

SMART REFRIGERATOR

SUBMITTED IN PARTIAL FULFILMENT OF THE

REQUIREMENT FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Submitted by

B. GAYATRI	**22RE1A0507**
B.MOUNIKA	**22RE1A0511**
B. PRASHANTH	**22RE1A0514**
K. RAGHAVA CHARY	**22RE1A0522**

Under the guidance of

Dr. Reddy Sekhar K

Assistant professor

School of Engineering & Technology



MNR COLLEGE OF ENGINEERING & TECHNOLOGY

(Approved by AICTE and Affiliated to JNT University, Hyderabad)

MNR NAGAR, FASALWADI, SANGAREDDY (dist) - 502001

2024-2025

MNR COLLEGE OF ENGINEERING & TECHNOLOGY

(Approved by AICTE and Affiliated to JNT University, Hyderabad)

MNR NAGAR, FASALWADI, SANGAREDDY(dist) - 502001

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

*This is to certify that the project work entitled "***SMART REFRIGERATOR"** *submitted by "***B.GAYATRI 22RE1A0507, B.MOUNIKA 22RE1A0511,B.PRASHANTH 22RE1A0514, K.RAGHAVA CHARY 22RE1A0522***"** *in partial fulfilment of the requirements for the award of the degree of* **Bachelor of Technology** *in* **COMPUTER SCIENCE AND ENGINEERING**, *by* **Jawaharlal Nehru Technological University Hyderabad during the academic year 2024-2025** *is a bonafide record of the work carried out under my guidance and superionion* at **MNR College of Engineering & Technology, Sangareddy.**

Internal guide	**Head of the department**
Dr. Reddy Sekhar. K	**Mr .T. Ravi kiran Kumar**
Assistant Professor School of Engineering &Technology	**Assistant professor**

External Examiner

ACKNOWLEDGEMENT

The successful completion of this project report would not have been possible without the guidance and support of several individuals and institutions. Gratitude is extended to **Sri M.S. Ravi Varma**, Vice-Chairman, for the encouragement and cooperation provided throughout the project duration.

His support was instrumental in facilitating various aspects of the work. Sincere appreciation is conveyed to **Dr. E.L. Nagesh**, Principal, **MNRCET**, for granting the necessary permissions and for ensuring a conducive environment to carry out the project work effectively.

Thanks are due to **Prof. T. Ravi Kiran Kumar**, Head of the Department, **CSE, MNR College of Engineering and Technology**, for his valuable support and motivation during this academic endeavor.

Special acknowledgment is given to **Dr. Reddy sekhar K**, Assistant Professor, **School of Engineering & Technology**, whose expert guidance, constant encouragement, and insightful feedback greatly contributed to the successful completion of the project. His mentorship played a crucial role in shaping the project from start to finish.

Appreciation is also extended to the faculty and staff members of the CSE department for their assistance and cooperation during various stages of the project.

This report is dedicated with deep respect and gratitude to everyone who contributed to the successful realization of this work.

In all sincerity,

B. GAYATRI	22RE1A0507
B. MOUNIKA	22RE1A0511
B. PRASHANTH	22RE1A0514
K. RAGHAVA CHARY	22RE1A0522

DECLARATION

We hereby declare that the Dissertation entitled **"SMART REFRIGERATOR"** which is being submitted in partial fulfilment of the requirement of the course leading to the award of the "**B. Tech** in **CSE**" in **MNR College of Engineering & Technology.** The result of this dissertation is carried out by our team, under the guidance and supervision of **Dr. Reddy sekhar K, Assistant Professor ,School of Engineering Technology**. We further declared that we have not previously submitted this to any other institution/university for any other degree.

B. GAYATRI	22RE1A0507
B. MOUNIKA	22RE1A0511
B. PRASHANTH	22RE1A0514
K. RAGHAVA CHARY	22RE1A0522

Table of Contents

1. Abstract 1
2. Introduction 2
3. Literature Survey 3
4. Existing System 4
4.1 Existing Algorithm 4
5.Proposed System.....7
5.1 Algorithms Used (Edge Detection, CNN) 7
6.System Requirements 9
6.1 Hardware Requirements 9
6.2 Software Requirements 9
7. System Analysis.....11
7.1 Feasibility Study ……...11
8. System Design 13
8.1 UML Diagrams (Use Case, Class, Sequence) 13
8.2 Collaboration Diagram …….....16
9. Implementation 17
9.1 Module Functions (Open Image, Recognize, Conversion) ……17
9.2 Source Code ……...……. 19
9.3 Result ……49
10. Software Environment ..... …. 51
10.1 Python Development Overview ..... …. 53
10.2 Libraries Used (TensorFlow, NumPy, Pandas, Matplotlib). 54
11. System Testing ..... …… 56
11.1 Types of tests(Unit, Integration, Functional) ……56
11.2 Test Cases 57

12. Conclusion 58
13. References 59

Smart refrigerator

ABSTRACT

The **Smart Refrigerator Using Barcode Scanner and Camera** is an innovative solution aimed at enhancing kitchen management through automation and smart technology. This system utilizes a barcode scanner to identify and log grocery items placed inside the refrigerator. Simultaneously, a camera captures visual data for object recognition, enabling the system to detect non-barcoded items such as fruits and vegetables.

The integrated system maintains an up-to-date inventory by combining barcode data with image processing techniques, providing real-time monitoring of stored items. Users are notified about expired or low-stock items through a connected application or interface.

our project introduces a Smart Refrigerator System equipped with a barcode scanner and an integrated camera, designed to efficiently manage both packed and unpacked food items while keeping track of their expiration dates.

This system utilizes a barcode scanner to quickly scan and identify packed food items. Once scanned, the system retrieves essential details such as product name and expiry date by decoding the barcode or accessing a predefined database. These packed items, along with their expiration details, are automatically stored in the central database for monitoring and management.

For unpacked or homemade food items, which typically lack barcodes, the system employs a camera interface to capture images of the items. Users are prompted to manually enter the item name and expiry date.

INTRODUCTION

In recent years, food waste has become a critical global issue, with millions of tons of edible food being discarded each year due to poor inventory management, lack of awareness, and improper storage. Households, which contribute significantly to this problem, often find themselves throwing away food that has expired simply because they forgot it was there or didn't realize it was nearing its expiration date.

our project introduces a Smart Refrigerator System that utilizes a barcode scanner and a camera to help users manage packed and unpacked food items efficiently, reduce waste, and enhance food safety.

The core idea of the project is to automate the process of food tracking inside a refrigerator by integrating hardware components such as a barcode scanner and a camera, along with a smart database and notification system. The goal is to build a system that not only stores and manages the list of food items but also monitors their expiration dates and proactively notifies users when items are about to expire. This helps in planning meals better, consuming items on time, and ultimately reducing food wastage.

Packed food items, which come with printed barcodes, can be easily scanned using a barcode scanner. These barcodes typically contain or reference key information such as the product name, batch number, and expiry date. Once a packed item is scanned, the system reads the barcode and retrieves the expiry details either directly from the barcode or from a predefined database where the product information is stored.

Literature Survey

Barcodes are a simple and efficient way to uniquely identify products. Almost all commercially packed food items come with a UPC (Universal Product Code) or EAN (European Article Number) barcode. These codes typically encode the product ID and, in some cases, batch or expiry information.

Kumar et al. (2016) developed a prototype smart fridge using a barcode scanner to detect items. Their system recorded the product name and associated expiry date from a predefined lookup table.

Park et al. (2014) created a kitchen management system that integrated barcode scanning with a food database. The application would update the inventory when items were added or removed and send reminders for items nearing expiry.

Unpacked items such as vegetables, fruits, dairy products without packaging, or homemade food lack barcodes. Tracking these items requires an image-based approach combined with manual inputs.

Amato et al. (2016) applied convolutional neural networks (CNNs) for recognizing food items in fridges. Their system could identify common foods like tomatoes, bananas, or milk cartons using image classification.

Xiao et al. (2017) proposed a smart fridge with built-in cameras to monitor contents continuously and generate item lists using deep learning.

Kim et al. (2020) improved food detection using YOLO (You Only Look Once) models for faster, more accurate object recognition in real-time.

Manual entry still plays a vital role in many systems, particularly for unpacked food or when expiry dates are not machine-readable. Many researchers acknowledge that while full automation is ideal, a semi-automated approach combining barcode/image input with manual date entry is more feasible.

Papadopoulos et al. (2015) proposed a smart kitchen assistant that used both manual entry and voice commands to record expiry dates and product names.

EXISTING SYSTEM

Several existing smart refrigerator systems have been developed or are currently in use, particularly in commercial and research environments. These systems aim to manage food inventory, reduce food wastage, and improve user convenience. However, most current systems either focus only on packed items using barcode scanning or are highly dependent on manual entry without full automation. Only a few advanced models attempt object detection for unpacked items using cameras and artificial intelligence. This section outlines these existing systems and the core algorithms they use.

Barcode-Based Inventory Systems

Example: Samsung Family Hub, LG Smart ThinQ

Function: These systems scan the barcodes of packed items to identify products.

Limitations:

Barcode data often does not include expiry dates, so users must enter them manually.

Cannot handle unpackaged or homemade items.

Limited intelligence and customization.

RFID-Based Smart Fridges

Example: MIT Smart Fridge Prototypes

Function: Items are tagged with RFID, and the fridge detects these tags to track inventory.

Limitations:

Requires RFID tagging of all items, which is not feasible for households.

Expensive and less flexible than barcode-based solutions.

Camera-Based Object Detection Systems

Example: Amato et al. (2016), Xiao et al. (2017)

Function: Uses computer vision (CV) to recognize items placed in the fridge using cameras.

Limitations:

Recognition accuracy is affected by lighting, occlusion, and item variety.

Requires advanced deep learning models and expensive hardware.

Manual Entry Systems with Expiry Tracking

Example: Fridge Pal App, Smart Pantry

Function: Users enter food items and expiry dates manually through an app.

Limitations:

Labor-intensive and inconvenient for daily use.

No automation or intelligence.

Barcode Parsing and Lookup Algorithms

Algorithm: UPC/EAN Parsing + Lookup Table or API Query

Working:

The barcode scanner reads the code.

A lookup table or online API (like Open Food Facts or Grocery APIs) is queried to retrieve product name and brand.

If expiry data is not included, the system asks the user to input it manually.

Limitations:

Relies heavily on third-party databases.

Not all barcodes have expiry date information.

Object Detection Algorithms (for unpacked items)

Algorithm: Convolutional Neural Networks (CNNs), YOLO (You Only Look Once), SSD (Single Shot Detector)

Working:

Image captured by a fridge-mounted camera is passed to a trained CNN.

The model classifies items based on trained food categories.

Detected items are added to the inventory with default or user-defined expiry durations.

Limitations:

Requires large lab led datasets.

Accuracy may vary based on image quality and item similarity.

Requires GPU for real-time detection.

c) Expiry Tracking Algorithm

Algorithm: Scheduled Expiry Checker

Working:

A background job (cronjob or cloud function) runs at regular intervals.

Compares current date with expiry date of each item in the database.

If the item is expired, it is removed from active inventory or moved to an “expired list.”

If the item is expiring within a threshold (e.g., 3 days), a notification is triggered.

Limitations:

Does not predict spoilage based on real conditions (e.g., temperature).

Depends on accurate expiry date input.

d) Notification and Alert Algorithm

Algorithm: Push Notification Trigger Logic

Working:

When expiry check algorithm flags an item, a notification is sent via:

Mobile app (using Firebase Cloud Messaging or similar)

Email

SMS (via services like Twilio)

Message includes item name, quantity, and days left.

Limitations:

Can become annoying if not configurable.

PROPOSED SYSTEM

The proposed smart refrigerator system aims to efficiently track and manage both packed and unpacked food items by leveraging a combination of barcode scanning, camera-based image processing, and intelligent algorithms. The system is designed to:

Scan packed items via a barcode scanner to extract product data.

Use a camera to capture images of unpacked items (like vegetables, fruits, dairy, etc.).

Allow users to manually enter expiry dates for unpacked items.

Store all item data in a centralized database.

Continuously check for expired or soon-to-expire items.

Send timely notifications to users.

To support this functionality, we propose the use of two primary algorithms:

1. Edge Detection Algorithm for pre-processing the camera input.

2.Convolutional Neural Network (CNN) for identifying and classifying unpacked food items (if automated recognition is extended in future scope).

System Architecture Overview.

The proposed system consists of the following components:

Barcode Scanner – Scans packed items.

Camera Module – Captures images of unpacked items.

Microcontroller/Processor – Controls peripherals and runs edge detection/CNN if supported (e.g., Raspberry Pi).

Database (Cloud or Local) – Stores item name, image, barcode, expiry date, and status.

Notification Module – Sends expiry alerts via app or message. Algorithms Used

A) Edge Detection Algorithm (For Image Preprocessing)

Purpose:

Edge detection is used as a preprocessing step for unpacked item images. It enhances image clarity by detecting boundaries of objects, improving the performance of the CNN or user manual identification.

Algorithm Used: (Edge Detection,CNN)

Steps:

1. Grayscale Conversion – Convert the image to grayscale to reduce complexity.

2. Gaussian Blur – Apply a Gaussian filter to reduce noise.

3. Gradient Calculation – Use Sobel operator to find intensity gradients.

4. Non-Maximum Suppression – Remove spurious response to edge detection.

5. Hysteresis Thresholding – Finalize edges based on high and low thresholds.

User Interface (Web/App) – Allows users to view inventory and manually add expiry dates.

Convolutional Neural Network (CNN) – For Food Image Classification

Purpose:

To automatically recognize the type of unpacked item (fruit, vegetable, etc.) based on its captured image.

Basic Architecture:

1. Input Layer – Takes the image as input (e.g., 128x128 pixels).

2. Convolutional Layers – Apply filters to detect low-level features (edges, curves).

3. ReLU Activation – Introduce non-linearity.

4. Pooling Layers – Reduce spatial dimensions (Max Pooling).

5. Flattening Layer – Converts 2D matrices to a 1D vector.

6. Fully Connected Layers – Interpret the image and classify it into a specific food category.

7. Output Layer – Gives predicted item label.

Dataset:

To train the CNN, a food image dataset is required. For prototyping, open datasets like:

Food-101

Fruits-360 can be use

SYSTEM REQUIREMENTS

This section outlines the essential hardware and software components required to build the Smart Refrigerator system using a camera and barcode scanner. The system aims to manage packed items through barcode scanning and unpacked items through image capture, storing all expiry data in a centralized database and notifying users of upcoming expirations.

HARDWARE REQUIREMENTS

Component Specification & Purpose.

Raspberry Pi 4 / 3B+ A Single Board Computer (SBC) to run the main application. Handles image capture, barcode scanning, and database communication.

USB Barcode Scanner A plug-and-play scanner used to read barcodes from packed food items. Helps automatically fetch product names and expiry details if encoded.

Camera Module USB webcam or Raspberry Pi Camera Module (5MP or higher). Captures images of unpacked food items for manual classification and expiry input.

Power Supply 5V/3A adapter to power the Raspberry Pi and peripherals like the barcode scanner and camera.

Storage MicroSD card (32GB or more) or SSD, used for OS, application code, and database storage.

Wi-Fi / EthernetFor database synchronization and sending user notifications about expiring products.

SOFTWARE REQUIREMENTS

Component Description.

Raspberry Pi OS / Ubuntu Lightweight OS for running Python applications, image processing, and device communication.

Python Main programming language for application logic, image handling, barcode scanning, and notifications.

OpenCV Used for image processing tasks such as edge detection and object segmentation from camera images.

Pyzbar / ZBar / ZXing Python libraries to decode barcodes from scanned images or input via barcode scanner.

SQLite / Firebase Local or cloud-based database to store product names, expiry dates, image metadata, and user data.

Tkinter / PyQt81 / Web App UI Simple GUI (if needed) to allow manual entry of expiry date for unpacked items after image capture.

Push Notification API Tools like Firebase Cloud Messaging (FCM) or SMTP (email) used to notify users of upcoming expiry dates.

Flask / Django (optional) Backend web framework to build a user interface or connect the app to a mobile/web-based front end.

Cron / Task Scheduler To periodically check the database and trigger expiry warnings or deletions.

Git / GitHub Version control system for source code management and collaborative development.

SYSTEM ANALYSIS

FEASIBILITY STUDY

The feasibility study helps to determine whether the proposed Smart Refrigerator system is practical and viable in terms of technology, cost, and usability. This project integrates a camera for unpacked food monitoring and a barcode scanner for packed item tracking, supported by a database and user notification mechanism. The feasibility of this system is analyzed under the following key areas:

1. **Technical Feasibility**

Technical feasibility evaluates whether the required technology is available and capable of supporting the system's design.

Barcode Scanning: Barcode scanners are readily available, affordable, and compatible with systems like Raspberry Pi or standard PCs. Libraries such as Pyzbar and ZBar support barcode decoding with high accuracy.

Camera Integration: USB webcams or Pi Cameras can be interfaced easily with Python and OpenCV for image capture. Manual entry of expiry dates after taking a photo ensures flexibility without relying on advanced AI.

Database Management: Lightweight databases like SQLite or Firebase can store the product list, expiry dates, and metadata efficiently with minimal setup.

Notification System: Email or push notifications can be implemented using APIs like Firebase Cloud Messaging or SMTP protocols.

System Architecture: Raspberry Pi (or any small-scale computing platform) can handle barcode scanning, image capture, database operations, and background tasks with moderate processing needs.

2. **Operational Feasibility**

Operational feasibility assesses how well the proposed system will function in the intended environment and whether end-users can operate it effectively.

Ease of Use: The system allows users to scan packed items easily and take photos of unpacked items. Manual entry of expiry dates is simple and intuitive.

Automation Level: The system automates expiry tracking and notification, reducing the need for constant human monitoring.

User Acceptance: Since the refrigerator enhances food management without complexity, users are likely to adopt it.

Maintenance: The system requires minimal user input after setup. Updating expiry dates or managing inventory is straightforward.

3.**Legal and Ethical Feasibility**

This aspect evaluates whether the system complies with data handling laws and ethical considerations.

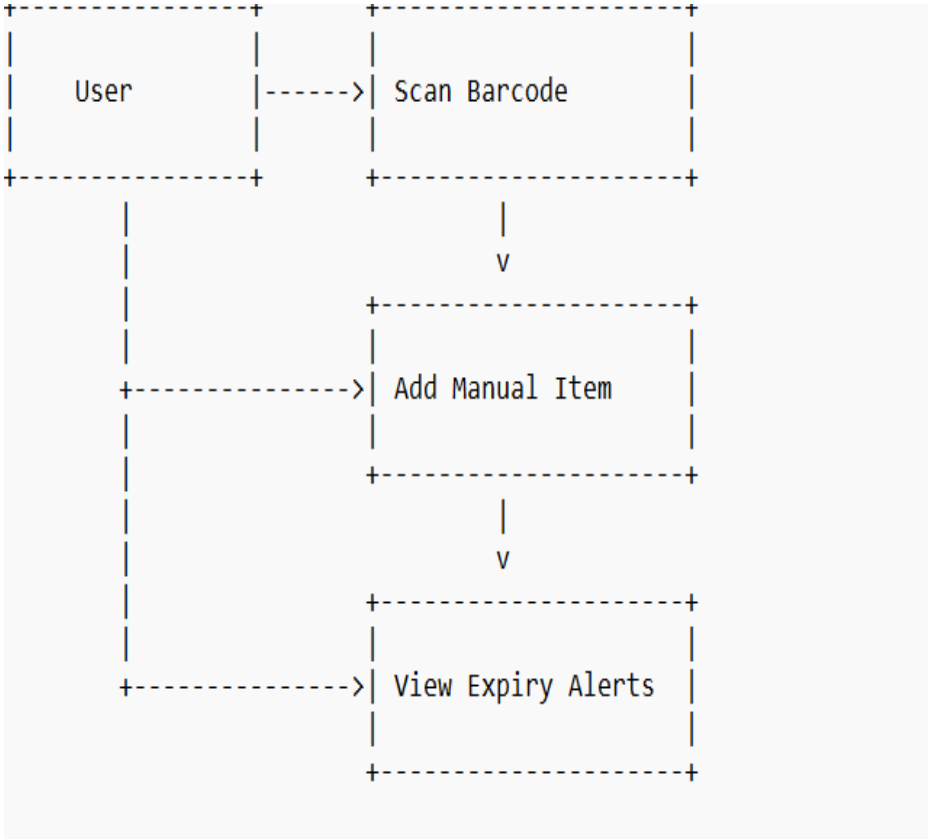
Data Privacy: The system stores expiry data and product names locally or in a secure database, with no sensitive personal data involved.

User Consent: As this is a personal or household system, users have full control over data input and access.

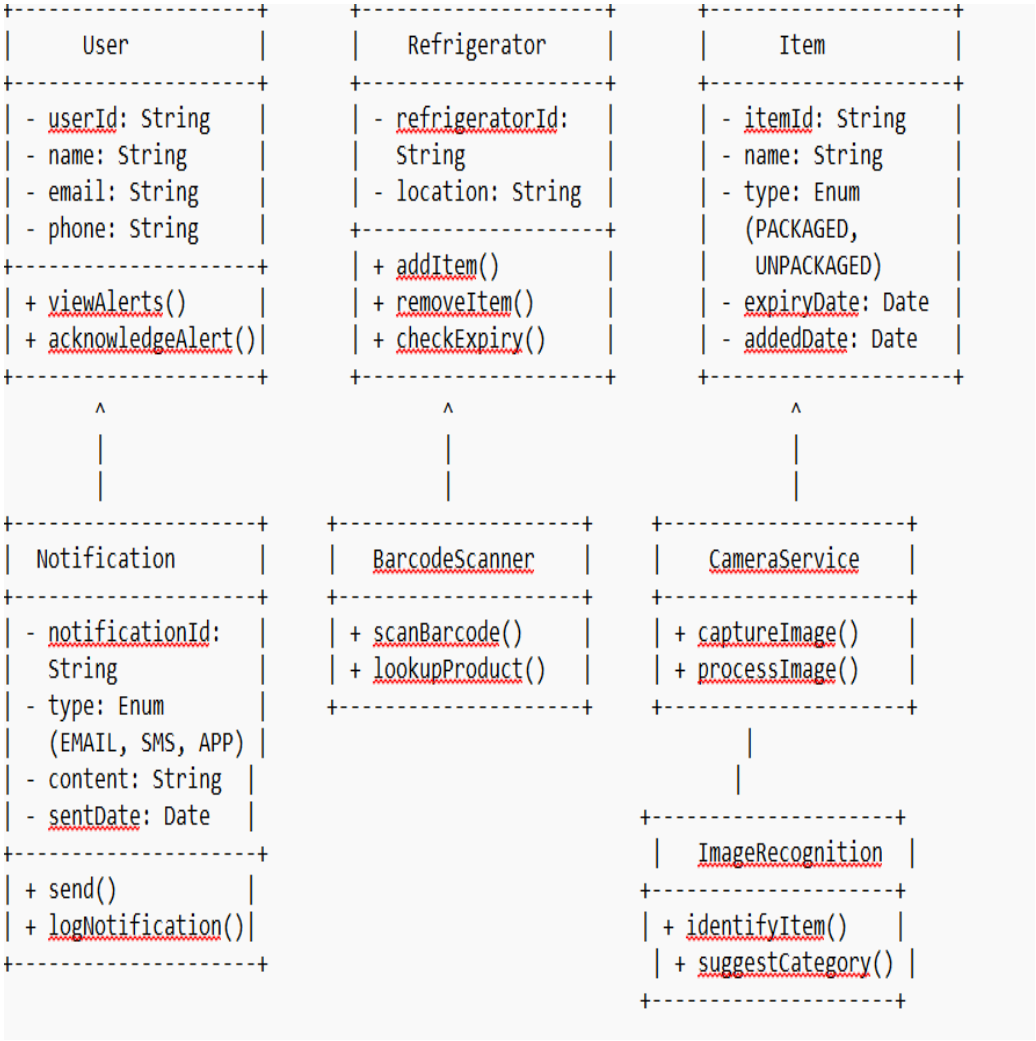
SYSTEM DESIGN

UML DIAGRAMS

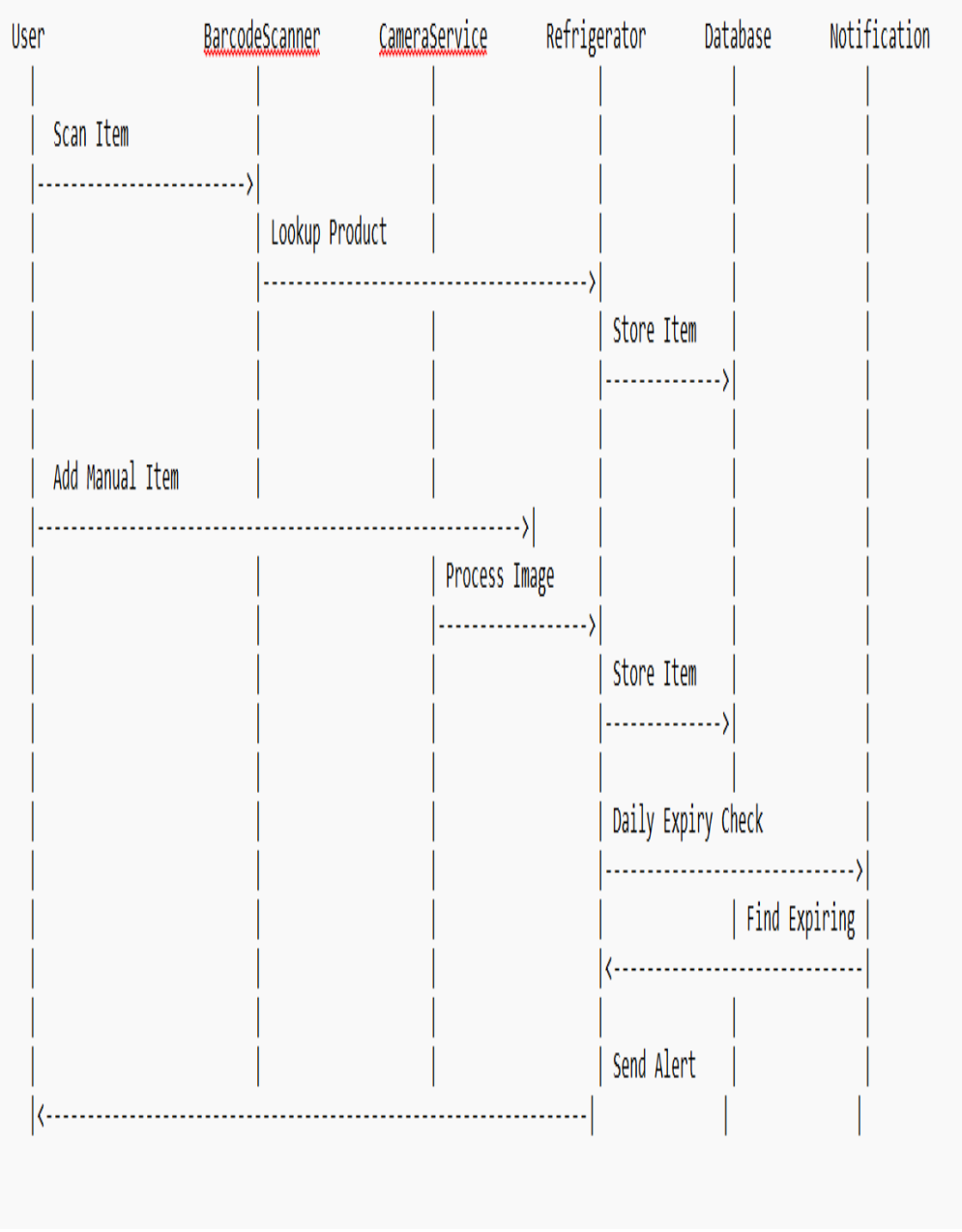
1.USE CASE DIAGRAM



2.CLASS DIAGRAM



3.SEQUENC DIAGRAM



4.**COLLABORATION DIAGRAM**



IMPLEMENTATION

MODULE FUNCTION:

1.**open image:**

Purpose: To allow the user to upload or capture an image of an unpacked item manually.

Functionality:

Opens the system file explorer or camera.

Lets the user select or take a picture of the unpacked food item.

The image is then displayed in the app GUI.

2.**Recognize**

Purpose: To recognize the food item in the image using computer vision (optional for MVP) and allow manual input of expiry.

Functionality:

Applies CNN (Convolutional Neural Network) or basic image preprocessing to identify item (if automated recognition is implemented).

Alternatively, allows the user to manually label the item name.

Prompts the user to manually enter expiry date.

3. **Conversion**

Purpose: To convert the recognized or entered data into structured format to store in the database.

Functionality:

Converts image input → data fields (item name, expiry date, image path).

For barcode items, converts scanned data → product details and expiry date (if encoded or fetched from external database).

Data saved in structured form (e.g., JSON or SQL database)

4. **Reset**

Purpose: To clear the current input or operation and start fresh.

Functionality:

Resets all fields on the current input form.

Clears uploaded image, text fields, and form controls.

Useful when incorrect image or data was added by mistake.

5.**Exit**

Purpose: To close the application safely.

Functionality:

Prompts confirmation from user: “Are you sure you want to exit?”

Safely saves or backs up any unsaved data (if implemented).

Closes GUI and releases all resources (camera, file handles, etc.).

SOURCE CODE

COMPONENTS

FOOD

Food item card.tsx

```
import React from 'react';
import { FoodItem } from '@/t1ypes';
import { format } from 'date-fns';
import{getFoodStatus,getDaysUntilExpiry}
from '@/services/foodItemService';
import { Trash2, Edit } from 'lucide-react';
import { Button } from '@/components/ui/button';
import { Badge } from '@/components/ui/badge';
import{Card,CardContent,CardFooter}
from '@/components/ui/card';
interface FoodItemCardProps {
  item: FoodItem;
  onEdit: (id: string) => void;
  onDelete: (id: string) => void;
}
constFoodItemCard=({item,
onEdit, onDelete }: FoodItemCardProps) => {
  const status = getFoodStatus(item.expiryDate);
  const daysUntilExpiry = getDaysUntilExpiry(item.expiryDate);
  const g etStatusDisplay = () => {
    if (status === 'expired') {
      return {
        bg: 'bg-expired-light',
```

```
        text: 'text-expired',
        badge: 'bg-expired',
        label: 'Expired',
      };
    } else if (status === 'expiring') {
      return {
        bg: 'bg-expiring-light',
        text: 'text-expiring',
        badge: 'bg-expiring',
        label: 'Expiring Soon',
      };
    } else {
      return {
        bg: 'bg-fresh-light',
        text: 'text-fresh',
        badge: 'bg-fresh',
        label: 'Fresh',
      };
    }
  };
  const statusDisplay = getStatusDisplay();
  return (
    <Card className={`${statusDisplay.bg} border-l-4 border-l-${status} mb-4 overflow-
hidden`}>
      <CardContent className="p-4">
        <div className="flex items-start justify-between">
          <div>
            <h3 className="font-bold text-lg mb-1">{item.name}</h3>
```

```
            <p className="text-sm text-gray-600">
              Expires: {format(item.expiryDate, 'MMM dd, yyyy')}
            </p>
            <div className="flex items-center mt-2">
              <Badge className={statusDisplay.badge}>
                {statusDisplay.label}
              </Badge>
              <span className="ml-2 text-sm font-medium">
                {daysUntilExpiry === 0
                  ? 'Today'
                  : daysUntilExpiry > 0
                  ? `${daysUntilExpiry} day${daysUntilExpiry === 1 ? '' : 's'} left`
                  : `${Math.abs(daysUntilExpiry)} day${Math.abs(daysUntilExpiry) === 1 ? ''
: 's'} ago`}
              </span>
            </div>
          </div>
          {item.imageUrl && (
            <div className="w-20 h-20 rounded-md overflow-hidden">
              <img
                src={item.imageUrl}
                alt={item.name}
                className="w-full h-full object-cover"
              />
            </div>
          )}
        </div>
      </CardContent>
```

```
      <CardFooter className="p-2 bg-white flex justify-end space-x-  2">
        <Button
          variant="outline"
          size="sm"
          onClick={() => onEdit(item.id)}
        >
          <Edit className="h-4 w-4 mr-1" /> Edit
        </Button>
        <Button
          variant="outline"
          size="sm"
          className="text-destructive hover:text-destructive"
          onClick={() => onDelete(item.id)}
        < <Trash2 className="h-4 w-4 mr-1" /> Delete
        </Button>
      </CardFooter>
    </Card>
  );
};
export default FoodItemCard.
```

Food item form.tsx

```
import React, { useState } from 'react';
import { use..Form } from 'react-hook-form';
import { z } from 'zod';
import { zodResolver } from '@hookform/resolvers/zod';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
```

```
import {
  Form,
  FormControl,
  FormField,
  FormItem,
  FormLabel,
  FormMessage,
} from '@/components/ui/form';
import { FoodItem } from '@/types';
import { format } from 'date-fns';
import { Camera } from 'lucide-react';
const formSchema = z.object({
  name: z.string().min(2, { message: 'Name must be at least 2 characters' }),
  expiryDate: z.string().refine((val) => !isNaN(Date.parse(val)), {
    message: 'Please enter a valid date',
  }),
  barcode: z.string().optional(),
  imageFile: z.any().optional(),
});
type FormData = z.infer<typeof formSchema>;
interface FoodItemFormProps {
  initialData?: Partial<FoodItem>;
  onSubmit: (data: Omit<FoodItem, 'id' | 'createdAt'>) => void;
  barcode?: string;
}
const FoodItemForm = ({ initialData, onSubmit, barcode }: FoodItemFormProps) => {
  const [imagePreview, setImagePreview] = useState<string | null>(initialData?.imageUrl
|| null);
```

```
const form = useForm<FormData>({
  resolver: zodResolver(formSchema),
  defaultValues: {
    name: initialData?.name || '',
    expiryDate: initialData?.expiryDate
      ? format(initialData.expiryDate, 'yyyy-MM-dd')
      : format(new Date(Date.now() + 7 * 24 * 60 * 60 * 1000), 'yyyy-MM-dd'),
    barcode: barcode || initialData?.barcode || '',
    imageFile: undefined,
  },
});
consthandleImageChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (file) {
    form.setValue('imageFile', file);
    const reader = new FileReader();
    reader.onloadend = () => {
      setImagePreview(reader.result as string);
    };
    reader.readAsDataURL(file);
  }
};
const handleSubmit = (data: FormData) => {
  const submissionData: Omit<FoodItem, 'id' | 'createdAt'> = {
    name: data.name,
    expiryDate: new Date(data.expiryDate),
    barcode: data.barcode || undefined,
    imageUrl: data. ImageFile
```

```
      ? imagePreview
      : initialData?.imageUrl,
  };
  onSubmit(submissionData);
};
return (
  <Form {...form}>
    <form onSubmit={form.handleSubmit(handleSubmit)} className="space-y-6">
      <FormField
        control={form.control}
        name="name"
        render={({ field }) => (
          <FormItem>
            <FormLabel>Item Name</FormLabel>
            <FormControl>
              <Input placeholder="Enter food item name" {...field} />
            </FormControl>
            <FormMessage />
          </FormItem>
        )}
      />
      <FormField
        control={form.control}
        name="expiryDate"
        render={({ field }) => (
          <FormItem>
            <FormLabel>Expiry Date</FormLabel>
            <FormControl>
```

```
            <Input type="date" {...field} />
          </FormControl>
          <FormMessage />
        </FormItem>
      )}
    />
    {barcode && (
      <FormField
        control={form.control}
        name="barcode"
        render={({ field }) => (
          <FormItem>
            <FormLabel>Barcode</FormLabel>
            <FormControl>
              <Input readOnly {...field} />
            </FormControl>
          </FormItem>
        )}
      />
    )}
    <div className="space-y-2">
      <Label HTML For="image">Item Image (Optional)</Label>
      <div className="flex items-center space-x-4">
        <Button
          type="button"
          variant="outline
onClick={() => document.getElementById('image-upload')?.click()}
          className="flex items-center"
```

```
          >
            <Camera className="mr-2 h-4 w-4" />
            {imagePreview ? 'Change Image' : 'Add Image'}
          </Button>
          <Input
            id="image-upload"
            type="file"
            accept="image/*"
            className="hidden"
            onChange={handleImageChange}
          />
        </div>
        {imagePreview && (
          <div className="mt-4 relative w-full max-w-md mx-auto">
            <img
              src={imagePreview}
              alt="Preview"
              className="rounded-md w-full h-48 object-cover"
            />
            <Button
              type="button"
              variant="secondary"
              size="sm"
              className="absolute top-2 right-2"
              onClick={() => {
                setImagePreview(null);
                form.setValue('imageFile', undefined);
              }}
```

```
            >
              Remove
            </Button>
          </div>
        )}
      </div>
      <Button type="submit" className="w-full">
        {initialData ? 'Update Item' : 'Add Item'}
      </Button>
    </form>
  </Form>
 );
};
export default FoodItemForm;
```

LAYOUT

Layout.tsx

```
import React from 'react';
import Navbar from './Navbar';
import { Toaster } from 'sonner';
interface LayoutProps {
 children: React.ReactNode;
}
const Layout = ({ children }: LayoutProps) => {
 return (
  <div className="min-h-screen flex flex-col">
   <Navbar />
   <main className="flex-1 container mx-auto p-4 pb-20 md:pb-4 max-w-4xl">
    {children}
```

```
      </main>
      <Toaster position="top-center" />
    </div>
  );
};
export default Layout;
```

NOTIFICATION

EXPIRY NOTIFICATION.TSX

```
import { useEffect, useState } from 'react';
import { FoodItem } from '@/types';
import { getAllFoodItems, getFoodStatus } from '@/services/foodItemService';
import { toast } from 'sonner';
import { Bell } from 'lucide-react';
import { useNavigate } from 'react-router-dom';
const ExpiryNotification = () => {
  const navigate = useNavigate();
  const [hasChecked, setHasChecked] = useState(false);
  useEffect(() => {
    // Only check once per component mount
    if (!hasChecked) {
      checkExpiringItems();
      setHasChecked(true);
    }
    // Check again when the component is focused
    const handleFocus = () => {
      checkExpiringItems();
    };
    window.addEventListener('focus', handleFocus);
```

```
  return () => {
    window.removeEventListener('focus', handleFocus);
  };
}, [hasChecked]);
const checkExpiringItems = () => {
  const items = getAllFoodItems();
  const expiringItems = items.filter(item =>
    getFoodStatus(item.expiryDate) === 'expiring'
  );
  if (expiringItems.length > 0) {
    showExpiryNotification(expiringItems);
  }
};
const showExpiryNotification = (items: FoodItem[]) => {
  toast(
    <div>
      <div className="font-semibold flex items-center">
        <Bell className="mr-2 h-4 w-4 text-orange-500" />
        Items Expiring Soon
      </div>
      <p className="text-sm mt-1">
        You have {items.length} item{items.length === 1 ? '' : 's'} expiring soon.
      </p>
    </div>,
    {
      duration: 5000,
      action: {
        label: "View Items",
```

```
        onClick: () => navigate('/dashboard'),
      },
    }
  );
 };
 return null; // This is just a utility component, no UI needed
};
export default ExpiryNotification
```

SCANNER

BARCODESCANNER

```
import React, { useEffect, useRef, useState } from 'react';
import { initBarcodeScanner, stopBarcodeScanner, onBarcodeDetected } from '@/utils/barcodeScanner';
import { Button } from '@/components/ui/button';
import { Camera, CameraOff } from 'lucide-react';
interface BarcodeScannerProps {
 onDetected: (barcode: string) => void;
}
const BarcodeScanner = ({ onDetected }: BarcodeScannerProps) => {
 const [isScanning, setIsScanning] = useState(false);
 const [error, setError] = useState<string | null>(null);
 const scannerRef = useRef<HTMLDivElement>(null);
 const startScanner = async () => {
  try {
   setError(null);
   await initBarcodeScanner('barcode-scanner');
   setIsScanning(true);
   onBarcodeDetected((barcode) => {
```

```
        stopBarcodeScanner().then(() => {
          setIsScanning(false);
          onDetected(barcode);
        });
      });
    } catch (err) {
      setError('Could not access camera. Please ensure camera permissions are granted.');
      console.error('Scanner error:', err);
      setIsScanning(false);
    }
  };
  const stopScanner = async () => {
    await stopBarcodeScanner();
    setIsScanning(false);
  };
  useEffect(() => {
    return () => {
      stopBarcodeScanner();
    };
  }, []);
  return (
    <div className="w-full flex flex-col items-center">
      <div
        id="barcode-scanner"
        ref={scannerRef}
          className="w-full max-w-md aspect-video bg-black relative overflow-hidden
rounded-lg my-4"
      >
```

```
        {!isScanning && (
         <div className="absolute inset-0 flex flex-col items-center justify-center bg-gray-
900 text-white p-4 text-center">
           <Camera size={48} className="mb-2" />
        <p>Press the button below to scan a barcode</p>
    {error && <p className="mt-2 text-red-40text-sm">{error}</p>}
         </div>
        )}
      </div>
      <div className="mt-4">
        {isScanning ? (
         <Button
           variant="outline"
           onClick={stopScanner}
           className="flex items-center"
         >
           <CameraOff className="mr-2 h-4 w-4" />
           Stop Scanning
         </Button>
        ) : (
         <Button
           onClick={startScanner}
           className="flex items-center"
         >
           <Camera className="mr-2 h-4 w-4" />
           Start Scanning
         </Button>
        )}
```

```
      </div>
    </div>
  );
};
export default BarcodeScanner;
```

PAGES

ADDITEM.TSX

```
import React from 'react';
import { useNavigate } from 'react-router-dom';
import FoodItemForm from '@/components/food/FoodItemForm';
import { addFoodItem } from '@/services/foodItemService';
import { FoodItem } from '@/types';
import { toast } from 'sonner';
import { Button } from '@/components/ui/button';
import { ArrowLeft } from 'lucide-react';
const AddItem = () => {
  const navigate = useNavigate();
  const handleFormSubmit = (data: Omit<FoodItem, 'id' | 'createdAt'>) => {
    addFoodItem(data);
    toast.success('Item added successfully!');
    navigate('/dashboard');
  };
  return (
    <div className="pb-10">
      <div className="flex items-center mb-6">
        <Button
          variant="ghost"
          size="sm"
```

```
        onClick={() => navigate(-1)}
        className="mr-2"
      >
        <ArrowLeft className="h-4 w-4" />
      </Button>
      <h1 className="text-2xl font-bold">Add Food Item</h1>
    </div>
    <p className="text-gray-600 mb-6">
     Add details about your food item below.
    </p>
    <FoodItemForm onSubmit={handleFormSubmit} />
   </div>
  );
};
export default AddItem;
```

DASHBOARD.TSX

```
import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import FoodItemCard from '@/components/food/FoodItemCard';
import { getAllFoodItems, deleteFoodItem, getFoodStatus } from '@/services/foodItemService';
import { FoodItem, FoodStatus } from '@/types';
import { toast } from 'sonner';
import { Button } from '@/components/ui/button';
import {
  Dialog,
  DialogContent,
  DialogHeader,
```

```
  DialogTitle,
  DialogDescription,
  DialogFooter,
  DialogClose
} from '@/components/ui/dialog';
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger
} from '@/components/ui/dropdown-menu';
import { ChevronDown, Plus, Trash2, Filter } from 'lucide-react';
importExpiryNotificationfrom '@/components/notifications/ExpiryNotification';
const Dashboard = () => {
  const navigate = useNavigate();
  const [items, setItems] = useState<FoodItem[]>([]);
  const [filteredItems, setFilteredItems] = useState<FoodItem[]>([]);
  const [filter, setFilter] = useState<FoodStatus | 'all'>('all');
  const[itemToDelete, setItemToDelete] = useState<string | null>(null);
  const[isDeleteDialogOpen,setIsDeleteDialogOpen] = useState(false);
  useEffect(() => {
    loadItems();
  }, []);
  useEffect(() => {
    if (filter === 'all') {
      setFilteredItems(items);
    } else {
      setFilteredItems(
```

```
      items.filter(item => getFoodStatus(item.expiryDate) === filter)
    );
  }
}, [filter, items]);
const loadItems = () => {
  const foodItems = getAllFoodItems();
  setItems(foodItems);
  setFilteredItems(foodItems);
};
const handleEditItem = (id: string) => {
  navigate(`/edit/${id}`);
};
const openDeleteDialog = (id: string) => {
  setItemToDelete(id);
  setIsDeleteDialogOpen(true);
};
const confirmDelete = () => {
  if (itemToDelete) {
    deleteFoodItem(itemToDelete);
    loadItems();
    toast.success('Item deleted successfully');
    setIsDeleteDialogOpen(false);
    setItemToDelete(null);
  }
};
const getFilterLabel = (filterValue: FoodStatus | 'all') => {
  switch (filterValue) {
    case 'fresh': return 'Fresh';
```

```
      case 'expiring': return 'Expiring Soon';
      case 'expired': return 'Expired';
      case 'all': default: return 'All Items';
    }
  };
  return (
    <div className="pb-10">
      <ExpiryNotification />
      <div className="flex flex-col md:flex-row md:items-center md:justify-between mb-
6">
        <h1 className="text-2xl font-bold mb-2 md:mb-0">Your Food Items</h1>
        <div className="flex space-x-2">
          <DropdownMenu>
            <DropdownMenuTrigger asChild>
              <Button variant="outline" className="flex items-center">
                <Filter className="h-4 w-4 mr-2" />
                {getFilterLabel(filter)}
                <ChevronDown className="h-4 w-4 ml-2" />
              </Button>
            </DropdownMenuTrigger>
            <DropdownMenuContent align="end">
              <DropdownMenuItem onClick={() => setFilter('all')}>
                All Items
              </DropdownMenuItem>
              <DropdownMenuItem onClick={() => setFilter('fresh')}>
                Fresh
              </DropdownMenuItem>
              <DropdownMenuItem onClick={() => setFilter('expiring')}>
```

```
            Expiring Soon
          </DropdownMenuItem>
          <DropdownMenuItem onClick={() => setFilter('expired')}>
            Expired
          </DropdownMenuItem>
        </DropdownMenuContent>
      </DropdownMenu>
      <Button onClick={() => navigate('/add')}>
        <Plus className="h-4 w-4 mr-2" />
        Add Item
      </Button>
    </div>
  </div>
  {filteredItems.length === 0 ? (
    <div className="text-center py-12">
      <p className="text-gray-500 mb-4">No items found</p>
      <Button onClick={() => navigate('/add')}>
        <Plus className="h-4 w-4 mr-2" />
        Add Your First Item
      </Button>
    </div>
  ) : (
    <div>
      {filteredItems.map((item) => (
        <FoodItemCard
          key={item.id}
          item={item}
          onEdit={handleEditItem}
```

```
            onDelete={openDeleteDialog}
          />
        ))}
      </div>
    )}
<Dialoopen={isDeleteDialogOpen} onOpenChange={setIsDeleteDialogOpen}>
      <DialogContent>
        <DialogHeader>
          <DialogTitle>Confirm Deletion</DialogTitle>
          <DialogDescription>
            Are you sure you want to delete this item? This action cannot be undone.
          </DialogDescription>
        </DialogHeader>
        <DialogFooter>
          <DialogClose asChild>
            <Button variant="outline">Cancel</Button>
          </DialogClose>
          <Button
            variant="destructive"
            onClick={confirmDelete}
            className="flex items-center"
          >
            <Trash2 className="h-4 w-4 mr-2" />
            Delete
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
```

```
    </div>
  );
};
export default Dashboard;
```

EDIT ITEM

```
import React, { useEffect, useState } from 'react';
import { useNavigate, useParams } from 'react-router-dom';
import FoodItemForm from '@/components/food/FoodItemForm';
import{updateFoodItem,getAllFoodItems}
from '@/services/foodItemService';
import { FoodItem } from '@/types';
import { toast } from 'sonner';
import { Button } from '@/components/ui/button';
import { ArrowLeft } from 'lucide-react';
const EditItem = () => {
  const navigate = useNavigate();
  const { id } = useParams<{ id: string }>();
  const [item, setItem] = useState<FoodItem | null>(null);
  useEffect(() => {
    if (id) {
      const allItems = getAllFoodItems();
      const foundItem = allItems.find(item => item.id === id);
      if (foundItem) {
        setItem(foundItem);
      } else {
        toast.error('Item not found');
        navigate('/dashboard');
      }
```

```
  }
 }, [id, navigate]);
 const handleFormSubmit = (data: Omit<FoodItem, 'id' | 'createdAt'>) => {
  if (id) {
   updateFoodItem(id, data);
   toast.success('Item updated successfully!');
   navigate('/dashboard');
  }
 };
 if (!item) {
  return <div>Loading...</div>;
 }
 return (
  <div className="pb-10">
   <div className="flex items-center mb-6">
    <Button
     variant="ghost"
     size="sm"
     onClick={() => navigate(-1)}
     className="mr-2"
    >
     <ArrowLeft className="h-4 w-4" />
    </Button>
    <h1 className="text-2xl font-bold">Edit Food Item</h1>
   </div>
<FoodItemFor initialData={item} onSubmit={handleFormSubmit} />
  </div>
 );
```

```
};
export default EditItem;
```

SCANBARCODE.TSX

```
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
importBarcodeScanner
from '@/components/scanner/BarcodeScanner';
import FoodItemForm from '@/components/food/FoodItemForm';
import { addFoodItem } from '@/services/foodItemService';
import { FoodItem } from '@/types';
import { toast } from 'sonner';
import { Button } from '@/components/ui/button';
import { ArrowLeft } from 'lucide-react';
const ScanBarcode = () => {
  const navigate = useNavigate();
  const [scannedBarcode, setScannedBarcode] = useState<string | null>(null);
  const handleBarcodeDetected = (barcode: string) => {
    setScannedBarcode(barcode);
    // In a real app, we would fetch product details from an external API here
    toast.success(`Barcode detected: ${barcode}`);
  };
  const handleFormSubmit = (data: Omit<FoodItem, 'id' | 'createdAt'>) => {
    addFoodItem(data);
    toast.success('Item added successfully!');
    navigate('/dashboard');
  };
  return (
    <div className="pb-10">
```

```
<div className="flex items-center mb-6">
  <Button
    variant="ghost"
    size="sm"
    onClick={() => navigate(-1)}
    className="mr-2"
  >
    <ArrowLeft className="h-4 w-4" />
  </Button>
  <h1 className="text-2xl font-bold">Scan Barcode</h1>
</div>
{!scannedBarcode ? (
  <div className="space-y-4">
    <p className="text-gray-600 mb-4">
      Position the barcode within the scanner area and hold steady.
    </p>
    <BarcodeScanner onDetected={handleBarcodeDetected} />
    <div className="mt-8 border-t pt-4">
      <Button
        variant="outline"
        className="w-full"
        onClick={() => navigate('/add')}
      >
        Skip and Add Manually
      </Button>
    </div>
  </div>
) : (
```

```
        <div>
          <p className="text-green-600 mb-6">
            Barcode scanned successfully! Please add item details below.
          </p>
<FoodItemForonSubmit={handleFormSubmit} barcode={scannedBarcode} />
        </div>
      )}
    </div>
  );
};
export default ScanBarcode;
```

App.css

```
#root {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}
.logo {
  height: 6em;
  padding: 1.5em;
  will-change: filter;
  transition: filter 300ms;
}
.logo:hover {
  filter: drop-shadow(0 0 2em #646cffaa);
}
.logo.react:hover {
```

```
  filter: drop-shadow(0 0 2em #61dafbaa);
}
@keyframes logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}
@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}
.card {
  padding: 2em;
}
.read-the-docs {
  color: #888;
}
```

APP.TSX

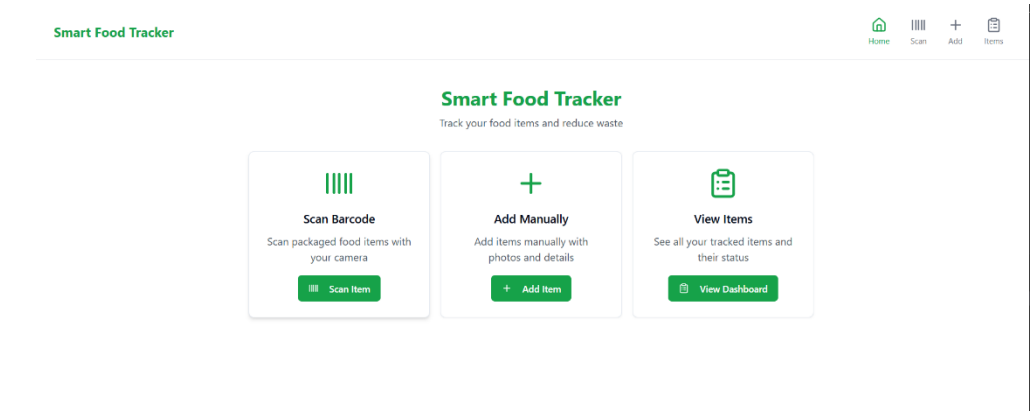
```
import { Toaster } from "@/components/ui/toaster";
import { Toaster as Sonner } from "@/components/ui/sonner";
import { TooltipProvider } from "@/components/ui/tooltip";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Layout from "./components/layout/Layout";
```

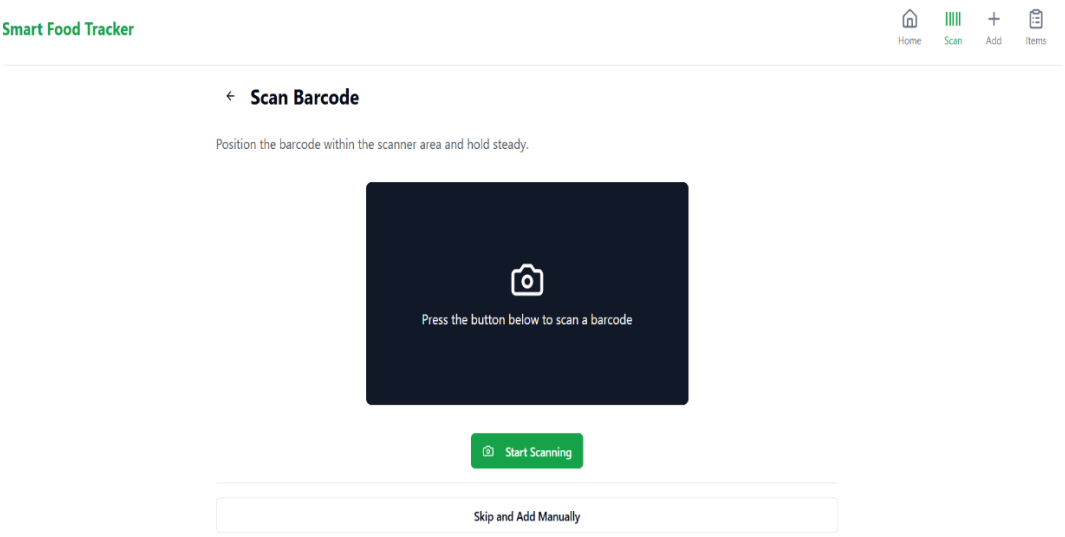
```
import Home from "./pages/Home";
import ScanBarcode from "./pages/ScanBarcode";
import AddItem from "./pages/AddItem";
import EditItem from "./pages/EditItem";
import Dashboard from "./pages/Dashboard";
import NotFound from "./pages/NotFound";
const queryClient = new QueryClient();
const App = () => (
  <QueryClientProvider client={queryClient}>
    <TooltipProvider>
      <Toaster />
      <Sonner />
      <BrowserRouter>
        <Layout>
          <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/scan" element={<ScanBarcode />} />
            <Route path="/add" element={<AddItem />} />
            <Route path="/edit/:id" element={<EditItem />} />
            <Route path="/dashboard" element={<Dashboard />} />
            <Route path="*" element={<NotFound />} />
          </Routes>
        </Layout>
      </BrowserRouter>
    </TooltipProvider>
  </QueryClientProvider>
);
export default App;
```

INDEX.HTML

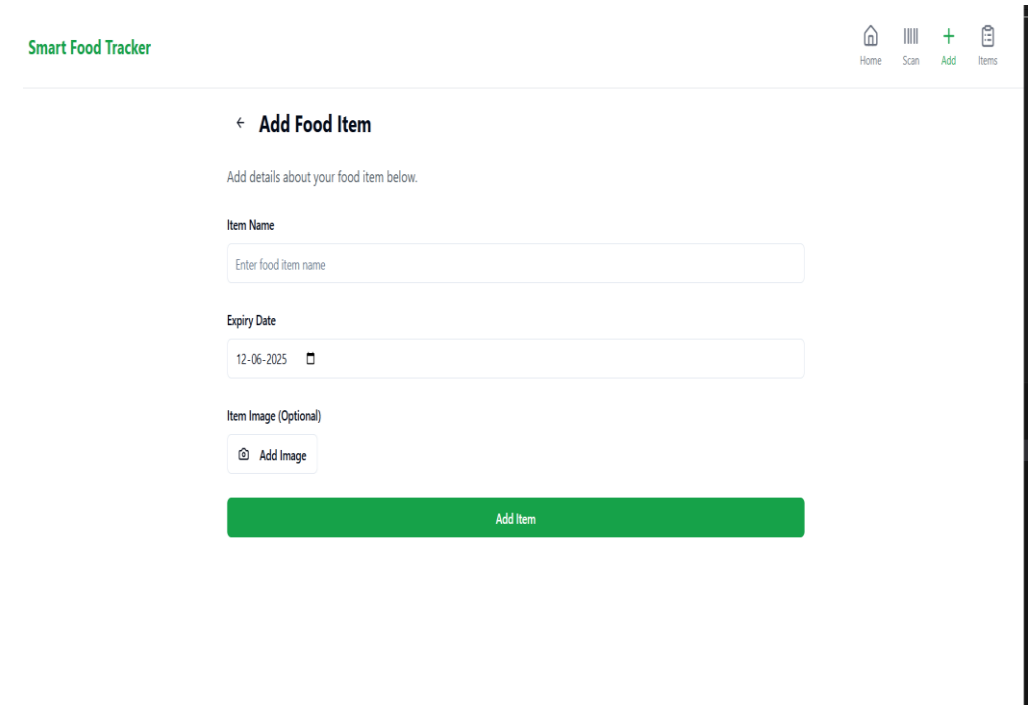
```
<!DOCTYPE html>More actions
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>food-savvy-tracker</title>
    <meta name="description" content="Lovable Generated Project" />
    <meta name="author" content="Lovable" />
    <meta property="og:title" content="food-savvy-tracker" />
    <meta property="og:description" content="Lovable Generated Project" />
    <meta property="og:type" content="website" />
<meta property="og:image" content="https://lovable.dev/opengraph-image-p98pqg.png"
/>
    <meta name="twitter:card" content="summary_large_image" />
    <meta name="twitter:site" content="@lovable_dev" />
<metaname="twitter:image"                content="https://lovable.dev/opengraph-image-
p98pqg.png" />
  </head>
  <body>
    <div id="root"></div>
     <!-- IMPORTANT: DO NOT REMOVE THIS SCRIPT TAG OR THIS VERY
COMMENT! -->
    <script src="https://cdn.gpteng.co/gptengineer.js" type="module"></script>
    <script type="module" src="/src/main.tsx"></script>
  </body>
<HTML>
```

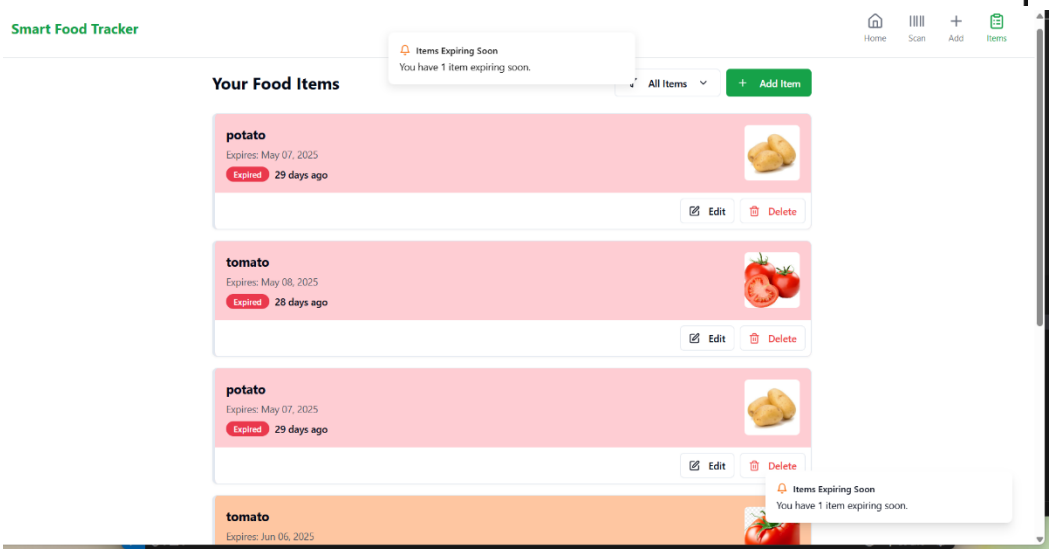
RESULT


Smart Food Tracker
Home
Scan
Add
Items
Smart Food Tracker
Track your food items and reduce waste
Scan Barcode
Scan packaged food items with your camera
Scan Item
Add Manually
Add items manually with photos and details
Add Item
View Items
See all your tracked items and their status
View Dashboard


Smart Food Tracker
Home
Scan
Add
Items
Scan Barcode
Position the barcode within the scanner area and hold steady.
Press the button below to scan a barcode
Start Scanning
Skip and Add Manually

Smart refrigerator





SOFTWARE ENVIRONMENT

For your smart refrigerator project that tracks expiry dates of both packed (via barcode scanner) and unpacked items (via camera + manual entry), here's a recommended software environment:

1. *Backend System*
   - *Node.js* with *Express.js* (lightweight and efficient for IoT applications)
   - *Python* with *Flask/Django* (alternative for more complex data processing)
   - *Database*:
     - *MongoDB* (flexible for varying item structures) or
     - *PostgreSQL* (if you need strict relational data)
2. *Frontend Interface*
   - *React.js* or *Vue.js* for web/mobile interfaces
   - *React Native* if building a dedicated mobile app
3. *Computer Vision Component*
   - *OpenCV* (for image processing of unpacked items)
   - *TensorFlow/PyTorch* (if implementing ML for food recognition)
4. *Barcode Processing*
   - *ZBar* or *QuaggaJS* (barcode scanning libraries)
   - Integration with product databases like Open Food Facts API
5. *Notification System*
   - *Firebase Cloud Messaging* (for mobile push notifications)
   - *Twilio* (for SMS alerts)
   - *Nodemailer* (for email notification)

Implementation Approach

1. *Barcode Items Flow*:
   - Scan barcode → Query product database → Extract expiry info → Store in database
2. *Unpacked Items Flow*:
   - Take photo → Manual expiry entry → (Optional) AI recognition → Store in database

3. *Notification System*:

   - Daily database scan for items nearing expiry

   - Threshold-based alerts (7 days, 3 days, expired)

   - Multiple notification channels

PYTHON DEVELOPMENT OVERVIEW

The smart refrigerator project is developed using Python, one of the most versatile and widely used programming languages in machine learning, image processing, and backend automation. Python provides a simple syntax, extensive libraries, and community support, which makes it ideal for integrating hardware (like a camera and barcode scanner) with intelligent software components (such as expiry tracking, database updates, and notification alerts).

The system integrates both GUI-based inputs (for user interaction with camera images and manual date entry) and backend automation (such as expiry checks and notifications). Python's support for object-oriented programming, data handling, and machine learning workflows ensures the system remains efficient, modular, and scalable.

Key development components include:

Camera access & image capture

Barcode scanning and decoding

Manual input for unpacked items

Database creation and maintenance

Expiry validation logic

Push notifications / alerts

Image processing (optional: future automation).

LIBRARIES USED:

Below are the primary Python libraries used in the implementation of the project:

1. TensorFlow

Purpose: Used for advanced deep learning and computer vision tasks.

Use in Project:

In future upgrades, TensorFlow can be integrated to automatically recognize food items from images using a CNN (Convolutional Neural Network).

Helpful in building models that identify food types from unpackaged items.

Features:

Supports GPU acceleration.

High-level API for model training and deployment.

Scalable from research to production.

2. NumPy

Purpose: Numerical processing and array manipulation.

Use in Project:

Handling and transforming arrays of pixel data from images captured via the camera.

Managing large sets of expiry-related data efficiently.

Features:

Fast matrix operations.

Ideal for data pre-processing for machine learning and image processing tasks.

3. Pandas

Purpose: Data analysis and manipulation tool.

Use in Project:

Used to manage and store food item data (name, barcode, expiry date, image reference, etc.) in DataFrames.

Allows filtering and searching for expired or soon-to-expire items.

Features:

Easy CSV/Excel data handling.

Simple interface for time-series operations (e.g., expiry countdown).

4. **Matplotlib**

Purpose: Data visualization library.

Use in Project:

Display item status (e.g., how many items are close to expiry, how many have expired).

Create visual dashboards or simple graphs in the GUI.

Features:

Rich plotting capabilities (bar charts, pie charts, line plots).

Customizable and embeddable in Python GUIs.

SYSTEM TESTING

In the context of the Smart Refrigerator, testing ensures that both packed (barcode-scanned) and unpacked (camera-captured) item features function accurately and that expiry detection and notifications operate as expected.

TYPES OF TEST:

1.**Unit Testing**

Unit testing is done at the developer level to verify individual components or modules of the system in isolation. Examples include:

Barcode scanning module

Image capture module

Expiry calculation logic

Notification module

2.**Integration Testing**

Integration testing checks the interaction between different modules to ensure they work together correctly. This is especially important for:

Camera integration with the expiry database

Barcode reader integration with item lookup and storage

Notification system working with expiry status

3.**Functional Testing**

This validates that the entire application performs according to the defined functional requirements:

Ability to add items using barcode or camera

Manual entry of expiry date for unpacked items

Automatic expiry tracking

Notification alerts

SIMPLE TEST CASES:

Test Case ID	Test Description	Input	Expected Output	Test Type	Status
TC001	Scan packed item with barcode	Barcode= "8901234567890"	Item details stored in DB with correct expiry	Functional	Pass/Fail
TC002	Capture image of unpacked item	Image file + expiry = "2025-06-10"	Image stored and expiry date saved	Functional	Pass/Fail
TC003	Check expiry status of item	Item with expiry = today	Item removed from list; notification triggered	Integration	Pass/Fail
TC004	Manual update of expiry date	Existing item, new date = "2025-07-01"	Updated in database	Unit	Pass/Fail
TC005	System sends expiry notification	Item expiring in <2 days	Alert sent to user	Integration	Pass/Fail

CONCLUSION

this smart refrigerator model is a step toward building a smart kitchen environment that promotes healthier living, minimizes food waste, and offers a user-friendly interface for daily household needs. It combines the best of both hardware (camera, barcode scanner) and software (image processing, database management, notifications) to deliver an effective and modern solution for food management.

This dual-approach allows the refrigerator to maintain a real-time database of all stored items, regardless of packaging. Packed items are quickly registered through barcode scanning, retrieving essential product details including the expiry date. For unpacked or homemade items, the user can simply capture an image and manually input the expiry information. Both types of data are stored in a centralized system, making it easy to monitor and manage.

The system automatically checks expiry dates and performs three major actions:

Notifies users about items that are approaching their expiration date.

Removes expired items from the database to maintain accuracy.

Promotes timely usage, thereby reducing food spoilage and economic loss.

REFERENCES

Food Reminder, retrieved on 12/23/11,

http://itunes.apple.com/us/app/food-reminder/id430227307

RZ fridge reminder for wp7, retrieved on 12/23/11,

http://www.windowsphoneapplist.com/rz_fridge_reminder-a62.htm

H. H. Wu, Y. T. ChuangLow-cost smart refrigerator(2017), pp. 228-231

View at publisherCrossrefView in ScopusGoogle Scholar

L. Xie, Y. Yin, X. Lu, B. Sheng, S. LuIfridge: an intelligent fridge for food management based on RFID technology

(2013), pp. 291-294

View at publisherCrossrefView in ScopusGoogle Scholar